# Guía paso a paso — Clase 6: Optimizadores

**Temas:** descenso por gradiente en 2D · mapas de contorno · momentum · decaimiento de la tasa de aprendizaje
**Duración:** ~2 h 30 min
**Material:** diapositivas `Clase6_Diapositivas` (PDF y PowerPoint, 21 diapositivas) · repositorio https://github.com/datahuba/curso-redes-neuronales
**Requisito:** Clase 3 (descenso por gradiente en una variable) y Clase 5
**Lectura base:** libro del Dr. Irving Vasquez, Optimización

## Objetivos de la clase

Al terminar, cada alumno podrá:

1. Aplicar el descenso por gradiente con dos parámetros e interpretar un mapa de contorno.
2. Explicar qué hace el **momentum** y cuándo ayuda.
3. Programar reglas de **decaimiento** de la tasa de aprendizaje.
4. Comparar optimizadores con experimentos y sacar conclusiones.

La guía tiene cuatro partes:

- **Parte A:** preparar las computadoras antes de clase.
- **Parte B:** teoría desde cero.
- **Parte C:** práctica en la computadora.
- **Parte D:** problemas frecuentes.

> **¿Por qué importa?** Entrenar una red es bajar por la "montaña del error" con miles de parámetros. El optimizador es la forma de caminar. Los que veremos hoy (y sus parientes, como Adam) son los que usan las librerías profesionales.

---

## Agenda

| Tiempo | Tema | Diapositivas |
|---|---|---|
| 0:00 | Presentación, objetivos y repaso (10 min) | 1–4 |
| 0:10 | B1. Descenso por gradiente en 2D (20 min) | 5–7 |
| 0:30 | C1. Práctica: notebook `21` (15 min) | 8 |
| 0:45 | B2. Valles estrechos y momentum (25 min) | 9–12 |
| 1:10 | C2. Práctica: notebook `22` (15 min) | 13 |
| 1:25 | Descanso (10 min) | 14 |
| 1:35 | B3. Decaimiento de la tasa de aprendizaje (25 min) | 15–18 |
| 2:00 | C3. Práctica: notebook `23` (20 min) | 19 |

| Tiempo | Tema | Diapositivas |
|---|---|---|
| 2:20 | Quiz y cierre (10 min) | 20–21 |

# PARTE A — Antes de clase (15 min)

En **Anaconda PowerShell Prompt**:

```
cd ~\projects\curso-redes-neuronales
```

```
git pull
```

```
conda activate nni
```

```
python docs/practica_06.py
```

**Verificación:** imprime tres bloques (esfera, momentum, decaimiento) y al final `Todas las comprobaciones pasaron.`

Con `--graficar` dibuja los cuatro caminos sobre la función de Beale. Úsalo en la clase.

**Checklist:**

- [ ] `python docs/practica_06.py --graficar` funciona.
- [ ] Ejecutaste `git pull`: el notebook `23_opt_lr_decay` es nuevo, porque el original estaba vacío.
- [ ] Sabes qué hacer si no hay ffmpeg: las animaciones de `21` y `22` se pueden guardar como GIF (ver Parte D).

# PARTE B — Teoría desde cero

## Repaso (5 min) — diapositivas 1–4

- "¿Cuál es la regla del descenso?" → $x \leftarrow x - \eta \cdot f'(x)$.
- "¿Qué pasa si $\eta$ es muy grande?" → oscila o diverge.
- "¿Y muy pequeña?" → avanza muy lento.

Hoy: qué hacer cuando hay **muchos** parámetros y la "montaña" tiene formas difíciles.

## B1. Descenso por gradiente en 2D (20 min) — diapositivas 5–7

### Derivadas parciales y gradiente (diapositiva 5)

Con dos variables, f(x, y), hay **una derivada por variable**. Se calcula como si la otra fuera un número fijo.

Ejemplo, la **esfera** $f(x, y) = x^2 + y^2$:

- $\partial f/\partial x = 2x$;
- $\partial f/\partial y = 2y$.

Juntas forman el **gradiente** $\nabla f = (2x, 2y)$: una flecha que apunta hacia donde la función **sube más rápido**.

**Regla:** cada parámetro baja en contra de su derivada:

- $x \leftarrow x - \eta \cdot \partial f/\partial x$;
- $y \leftarrow y - \eta \cdot \partial f/\partial y$.

### Mapa de contorno (diapositiva 6)

Es como un **mapa topográfico**: cada línea (o color) une puntos de la misma "altura". En la esfera son círculos alrededor del mínimo (0, 0).

Un camino de descenso cruza las líneas **en ángulo recto**, porque el gradiente siempre es perpendicular a ellas.

### El efecto de η en 2D (diapositiva 7)

Esfera, desde $(-1, -1)$, máximo 300 iteraciones. Son resultados del notebook `21`:

| η | Iteraciones | Qué pasa |
|---|---|---|
| 0.01 | 300 | Lento: termina en $(-0.0023, -0.0023)$, sin llegar |
| 0.1 | 56 | Bien |
| 0.5 | 2 | De un salto (solo en esta función) |
| 0.9 | 66 | Llega zigzagueando |
| 1.0 | 300 | Rebota para siempre entre $(-1, -1)$ y $(1, 1)$ |

**¿Por qué?** Cada paso multiplica x por $(1 - 2\eta)$:

- con $\eta = 0.9$, ese factor es $-0.8$: cambia de signo y se achica;
- con $\eta = 1$, es $-1$: cambia de signo y nunca se achica;
- con $\eta > 1$, crece y diverge.

## B2. Valles estrechos y momentum (25 min) — diapositivas 9–12

### El problema (diapositiva 9)

La **función de Beale** tiene un valle **largo, estrecho y curvo**, con el mínimo en (3, 0.5). En valles así, el gradiente apunta casi siempre **hacia las paredes**, no a lo largo del valle. Con η grande, el descenso **rebota** de pared a pared y puede salir disparado.

### La idea del momentum (diapositiva 10)

En lugar de seguir solo el gradiente de este instante, se usa un **promedio** de los gradientes recientes:

- $m \leftarrow \beta \cdot m + (1 - \beta) \cdot \nabla f$
- parámetros ← parámetros $- \eta \cdot m$

**β** (beta, entre 0 y 1) es la "memoria". Con β = 0.9, el paso de ahora se parece mucho a los anteriores.

### Intuición (diapositiva 11)

**Analogía:** una bola que rueda por el valle. Los empujones hacia los lados (las paredes) cambian de signo en cada rebote y **se cancelan** en el promedio. El empuje a lo largo del valle va siempre en la misma dirección y **se mantiene**.

| β | Efecto |
|---|---|
| 0 | Sin memoria: es el descenso simple |
| 0.5–0.9 | Suaviza los rebotes (lo habitual) |
| 0.99 | Demasiada memoria: le cuesta frenar y girar |

### Resultados (diapositiva 12)

Beale desde (2.5, 1.0), 500 iteraciones. Son resultados del notebook `22` y de `practica_06.py`:

| η | Descenso simple | Momentum β = 0.5 | Momentum β = 0.9 |
|---|---|---|---|
| 0.01 | $f = 6.95 \cdot 10^{-4}$ | $f = 7.37 \cdot 10^{-4}$ | $f = 1.07 \cdot 10^{-3}$ |
| **0.03** | **diverge** | $f = 1.9 \cdot 10^{-6}$ | **$f = 8.0 \cdot 10^{-7}$** |

**Conclusión honesta:**

- Con η pequeña, los tres llegan casi igual. Esta versión de momentum **promedia**: no acelera los pasos.
- Su ventaja es que **permite usar una η más grande** sin que el entrenamiento se descontrole, y así llega mucho más cerca del mínimo.

---

## B3. Decaimiento de la tasa de aprendizaje (25 min) — diapositivas 15–18

### El dilema (diapositiva 15)

Con η **fija**:

- si es grande, avanza rápido pero, cerca del mínimo, rebota;
- si es pequeña, afina bien pero tarda muchísimo.

**Solución:** empezar grande y **reducirla** con el tiempo.

### Tres reglas (diapositiva 16)

t es el número de iteración.

| Regla | Fórmula | Idea |
|---|---|---|
| Escalonada | $\eta_0 \cdot \gamma^{(t // s)}$ | Cada s iteraciones multiplica por γ (por ejemplo, a la mitad) |
| Inversa | $\eta_0 / (1 + k \cdot t)$ | Baja rápido al inicio, luego despacio |
| Exponencial | $\eta_0 \cdot e^{(-k \cdot t)}$ | Baja suave y continua |

### Calcular a mano (diapositiva 17)

Con $\eta_0 = 0.05$, $\gamma = 0.5$, $s = 100$ y $k = 0.01$:

| t | Escalonada | Inversa | Exponencial |
|---|---|---|---|
| 0 | 0.05 | 0.05 | 0.05 |
| 100 | 0.025 | 0.025 | 0.0184 |
| 250 | 0.0125 | 0.0143 | 0.0041 |

Haz en el pizarrón la escalonada en t = 250: 250 // 100 = 2, y $0.05 \cdot 0.5^2 = 0.0125$.

### Resultados (diapositiva 18)

Beale desde (1.0, 1.0), $\eta_0 = 0.05$, 500 iteraciones. Son resultados del notebook `23`:

| Configuración | f final |
|---|---|
| Tasa fija 0.05 | $5.8 \cdot 10^{-2}$ (rebota, no llega) |
| Escalonada | **$6.3 \cdot 10^{-5}$** |
| Inversa | $1.0 \cdot 10^{-4}$ |
| Exponencial | $1.5 \cdot 10^{-3}$ (decae demasiado rápido y se "congela") |
| Escalonada con $\gamma = 0.8$ | **$7.8 \cdot 10^{-7}$** |

**Mensaje:**

- El decaimiento escalonado da casi **mil veces** menos error que la tasa fija.
- Si la tasa decae **demasiado** rápido, el optimizador se queda quieto antes de llegar.

---

# PARTE C — Práctica en la computadora

Guías celda por celda: `docs/guias_notebooks/` (NB_21, NB_22 y NB_23).

## C1. `notebooks/21_opt_gradiente_2d.ipynb` (15 min, experimentos) — diapositiva 8

- Ejecuta todo con los valores originales.
  ✅ **Comprobación:** termina en ≈ (−0.00233, −0.00233), con f ≈ $1.09 \cdot 10^{-5}$.
- Cambia `learning_rate` a 0.1, 0.5, 0.9 y 1.0, y compara con la tabla de B1.
- La animación necesita ffmpeg. Si no está, usa `.save('gradient_descent.gif', writer='pillow')`.

## C2. `notebooks/22_opt_momentum.ipynb` (15 min, experimentos) — diapositiva 13

- Ejecuta con los valores originales.
  ✅ **Comprobación:** termina en ≈ (2.922, 0.480), con f ≈ $1.07 \cdot 10^{-3}$.
- Sube `learning_rate` a 0.03 y compara con el descenso simple: quita el `#` de la línea comentada.
  ✅ **Comprobación:** el descenso simple diverge ( `OverflowError` ) y el momentum llega a ≈ (2.998,

0.499).

- **Crea un optimizador nuevo** en cada prueba: el objeto recuerda su momentum.

### C3. `notebooks/23_opt_lr_decay.ipynb` (20 min, 3 puntos) — diapositiva 19

- **TODO 1:** las tres reglas.
  ✅ **Comprobación:** los valores de la tabla de B3.
- **TODO 2:** `next_params` con la tasa de la iteración actual.
  ✅ **Comprobación:** tasa fija f = $5.76 \cdot 10^{-2}$, contra decaimiento f = $6.34 \cdot 10^{-5}$.
- **TODO 3:** experimentos de la tabla de resultados.

**Demo final:**

```
python docs/practica_06.py --graficar
```

Muestra los cuatro caminos: el descenso lento, el momentum con η grande, la tasa fija rebotando y el decaimiento llegando al mínimo.

---

## Quiz y cierre (10 min) — diapositivas 20–21

| Pregunta | Respuesta |
|---|---|
| ¿Qué es el gradiente de f(x, y)? | El vector de derivadas parciales; apunta hacia donde f sube más rápido |
| ¿Por qué el descenso rebota en un valle estrecho? | El gradiente apunta a las paredes, no a lo largo del valle |
| ¿Qué controla β en el momentum? | Cuánta memoria de los gradientes anteriores tiene |
| ¿Qué ventaja dio el momentum en Beale? | Permitió una η grande (0.03) sin diverger |
| ¿Qué pasa si la tasa decae demasiado rápido? | Se "congela" antes de llegar al mínimo |

**Para llevar:**

1. Más parámetros = mismo descenso, una derivada por parámetro.
2. El momentum suaviza los rebotes y permite tasas más grandes.
3. El decaimiento empieza rápido y termina fino.

**Tarea:**

1. Terminar el notebook `23` (3 puntos) y entregar los experimentos de `21` y `22` con una conclusión.
2. En `docs/practica_06.py`, prueba el momentum con β = 0.99 y η = 0.03: ¿qué pasa?
3. Investigar en una frase qué es el optimizador **Adam**: combina momentum con una tasa que se adapta a cada parámetro.

**Rúbrica:**

| Notebook | Puntos |
|---|---|
| 21 (experimentos y conclusión) | 1 |

| Notebook | Puntos |
|---|---|
| 22 (experimentos y conclusión) | 1 |
| 23 | 3 |

## PARTE D — Problemas frecuentes

| Síntoma | Causa | Solución |
|---|---|---|
| `RuntimeError: ... MovieWriter (ffmpeg) not available` | La animación necesita ffmpeg | Cambia a `.save('archivo.gif', writer='pillow')`, o salta esa celda |
| `OverflowError: (34, 'Result too large')` | El descenso diverge | Baja η, o usa momentum |
| Aviso `Locator attempting to generate 1001 ticks` | `levels=1000` en el contorno | Solo es un aviso; puedes bajar a 100 |
| Resultados raros al repetir una prueba | Reusaste el optimizador (guarda su momentum o su contador t) | Crea uno nuevo |
| La tasa en `23` no cambia | Falta `self.t += 1` | Incrementa el contador en `next_params` |
| `AttributeError: module 'nni' has no attribute ...` | Notebook fuera del repo, o `nni` viejo | Abre la carpeta del repo y haz `git pull` |